

JobPilot：智能简历生成与求职投递管理系统

0. 文档说明

本文档用于《面向对象技术与方法》结课设计项目启动与开发执行。文档覆盖项目说明、需求说明、功能分解、系统设计、AI 服务设计、数据库设计、接口设计、云服务器部署说明、项目管理计划、四人小组分工，以及每名成员可直接执行的工作文档。

项目定位为面向求职者的智能化求职工作台，不做企业招聘平台，不要求企业或单位入驻发布岗位。系统通过“简历生成与解析、岗位导入与解析、岗位匹配、简历优化、模拟面试、自动投递辅助、投递进度管理、求职统计报告”形成完整闭环。

本项目后端统一采用 **Python + FastAPI** 技术栈，直接部署在云服务器上，暂不使用 Docker 打包。前端只保留一套响应式 Web，适配桌面端、平板和手机浏览器。

第一部分：完整项目说明文档

1. 项目名称

JobPilot：智能简历生成与求职投递管理系统

可选中文名称：

面向高校毕业生的智能简历生成、岗位适配与自动投递管理系统

2. 项目背景

高校毕业生和普通求职者在求职过程中需要反复完成简历撰写、岗位筛选、简历针对岗位修改、招聘系统重复填表、面试准备、投递状态跟踪和求职复盘等工作。这些任务高度依赖文本处理、信息抽取、结构化数据管理和流程管理，适合通过大语言模型、OCR、结构化简历建模和自动化填表辅助实现效率提升。

现有简历工具通常偏重模板排版，招聘平台偏重岗位发布与沟通，通用 AI 工具虽然能生成文本，但缺少长期保存的简历版本、岗位库、投递记录、面试记录和求职统计能力。本系统将用户简历、岗位信息、AI 分析报告、投递进度和求职统计统一建模，面向求职者提供一套可持续使用的个人求职管理平台。

3. 项目目标

本项目目标是开发一个可部署、可演示、可维护、功能完整的智能化移动互联系统。系统包含响应式 Web 前端、Python FastAPI 后端、MySQL 数据库、Redis 缓存、文件存储、OCR 服务、AI 服务、Playwright 自动填表辅助服务、日志监控和云服务器部署脚本。

系统目标包括：

1. 用户可以通过自然语言描述生成结构化简历。
2. 用户可以上传 PDF、DOCX、图片版 PDF 或简历截图，系统解析为可编辑的结构化简历。
3. 用户可以使用系统内置模板或上传个人模板渲染简历，并导出 PDF。
4. 用户可以从文本、链接、文件、截图等来源导入岗位信息。
5. 系统可以解析岗位 JD，提取岗位职责、任职要求、技能关键词和投递信息。
6. 系统可以计算简历与岗位的匹配度，并生成匹配报告。
7. 系统可以根据岗位要求生成岗位适配版简历。
8. 系统可以对简历进行表达优化、关键词补强、项目经历改写建议和风险提示。
9. 系统可以根据简历和岗位生成模拟面试题，并评价用户回答。
10. 系统可以维护标准投递信息，辅助用户在不同单位招聘系统中重复填写相同信息。
11. 系统可以跟踪投递状态，形成投递看板。
12. 系统可以统计求职数据并生成 AI 求职报告。

4. 系统用户角色

4.1 求职者

求职者是系统核心用户。求职者可以生成简历、上传解析简历、上传简历模板、导入岗位、进行岗位匹配、生成岗位适配简历、优化简历、进行模拟面试、创建自动投递任务、管理投递进度、查看数据统计和求职报告。

4.2 系统管理员

系统管理员负责用户管理、系统模板管理、Prompt 模板管理、AI 调用日志管理、OCR 任务日志管理、自动投递站点配置、系统参数配置和运行日志查看。

5. 领域边界

系统属于智能化求职辅助与个人职业管理领域，主要服务于求职者侧。

系统覆盖范围：

1. 简历生成与解析。
2. 图片版 PDF 与截图 OCR 识别。
3. 简历结构化编辑与版本管理。
4. 系统模板和用户自定义模板管理。
5. 岗位信息导入、解析、结构化和管理。
6. 简历与岗位 JD 的匹配分析。
7. 简历岗位适配与简历优化。
8. 模拟面试与回答评价。
9. 标准投递信息管理和自动填表辅助。
10. 投递记录和投递状态管理。
11. 求职统计与 AI 求职报告。

系统不覆盖范围：

1. 不作为真实招聘平台，不要求单位在系统内发布岗位。
2. 不保证外部岗位真实性，岗位信息主要由用户主动导入。
3. 不绕过验证码、短信验证、扫码登录、反爬机制和网站安全限制。
4. 不保存用户第三方招聘网站密码。
5. 不进行批量骚扰式投递。
6. 不代替用户做最终投递决策，自动投递前必须由用户确认。
7. 不伪造用户经历，不生成虚假学历、虚假项目、虚假实习或虚假证书。

6. 平台可用性说明：岗位信息从哪里来

本系统不依赖企业入驻发布岗位，而是作为求职者侧工具，帮助用户将外部零散岗位信息导入、解析、匹配和管理。求职者真实求职时，岗位来源通常分散在学校就业网、企业官网、招聘平台、公众号推文、邮件通知、微信群消息、PDF 招聘简章、宣讲会海报和同学转发链接中。本系统的价值在于把这些分散信息转化为可检索、可匹配、可投递、可统计的数据。

岗位信息来源包括：

1. 手动录入：用户填写公司、岗位名、城市、JD、截止时间和投递链接。
2. 文本粘贴导入：用户复制外部岗位描述，系统用 AI 解析为结构化岗位。

3. 公开 URL 尝试解析：用户粘贴招聘页面链接，系统尝试抓取公开页面正文并解析。若页面需要登录、禁止访问或解析失败，则提示用户改用文本粘贴或截图导入。
4. 文件导入：用户上传 PDF、DOCX、TXT 招聘简章，系统提取文本并解析岗位。
5. 截图导入：用户上传岗位截图或宣讲会海报，系统 OCR 识别后解析岗位。
6. 批量文本导入：用户一次性粘贴多个岗位信息，系统拆分并批量保存。

本系统定位为 **个人求职 CRM / AI 求职工作台 / 简历岗位匹配投递助手**，而不是招聘信息发布平台。

7. 核心业务闭环

1. 用户注册并登录系统。
2. 用户通过文字描述生成初始简历，或上传已有简历文件生成结构化简历。
3. 对图片版 PDF 或截图简历，系统先 OCR，再调用 AI 解析为结构化简历。
4. 用户在简历编辑器中修改教育经历、项目经历、实习经历、技能、获奖经历等内容。
5. 用户选择系统模板或上传个人模板，系统渲染简历并导出 PDF。
6. 用户通过文本、URL、文件或截图导入目标岗位。
7. 系统解析岗位 JD，提取职责、要求、关键词、投递链接和截止时间。
8. 系统计算简历与岗位匹配度，并生成匹配报告。
9. 用户生成岗位适配版简历。
10. 用户对简历进行 AI 优化。
11. 用户发起模拟面试，系统生成面试题并评价回答。
12. 用户创建投递任务，系统辅助自动填表或生成复制清单。
13. 系统保存投递记录并跟踪状态流转。
14. 系统统计投递进展、岗位来源、岗位分布、匹配度和面试转化率，并生成求职报告。

8. 功能分解

8.1 用户与权限模块

权重：8。

功能包括用户注册、登录、JWT 鉴权、密码加密、用户资料维护、角色管理、管理员用户管理、登录日志、账号启用禁用和前端路由权限控制。

依赖关系：所有业务模块依赖用户登录和权限校验。

8.2 文字生成简历模块

权重：10。

用户输入自然语言描述，选择目标岗位方向和简历风格，系统调用 LLM 生成结构化简历草稿，并保存为 ResumeVersion。用户可以继续在结构化简历编辑器中修改。

8.3 简历文件解析模块

权重：10。

支持 PDF、DOCX、图片版 PDF、PNG、JPG、JPEG。系统先提取文本；若 PDF 为图片版或扫描件，则先转图片并 OCR；之后调用 LLM 将文本解析为结构化简历。

Python 技术方案：

1. DOCX：python-docx。
2. 文本型 PDF：PyMuPDF、pdfplumber 或 pypdf。
3. 图片版 PDF：PyMuPDF 将页面渲染为图片。
4. 图片 OCR：PaddleOCR 优先，Tesseract OCR 或云 OCR 作为备用。
5. 结构化解析：LLM 输出 JSON，再由 Pydantic 校验。

8.4 简历模板管理与模板更换模块

权重：8。

功能包括系统内置模板、模板预览、模板切换、字段显示控制、字号和间距配置、HTML 渲染、PDF 导出、管理员模板管理、用户上传个人模板、用户模板私有可见、用户复制系统模板并修改样式。

用户上传模板建议格式：

1. HTML + CSS 模板。
2. 模板中使用占位符，例如 {{basic_info.real_name}}、{{education_list}}、{{project_list}}。
3. 用户模板禁止上传 script、iframe 和外链 JS。
4. 后端使用 Jinja2 渲染模板。

8.5 岗位信息导入与管理模块

权重：10。

功能包括手动创建岗位、文本粘贴导入、URL 尝试解析、文件导入、截图 OCR 导入、批量导入、岗位去重、岗位标签、岗位收藏、岗位搜索与筛选、岗位来源记录。

8.6 岗位匹配模块

权重：10。

系统基于简历结构化数据和岗位要求计算匹配度，并调用 AI 生成自然语言解释。

评分建议：

1. 技能关键词覆盖率：40%。
2. 项目经历相关度：25%。
3. 教育背景与学历要求：10%。
4. 实习或实践经历相关度：10%。
5. 求职意向、城市、岗位类型匹配：10%。
6. 简历完整度：5%。

8.7 简历岗位适配模块

权重：10。

系统根据目标岗位 JD 自动调整技能排序、突出相关项目经历、改写项目描述、生成岗位适配版求职意向和自我评价，并保存为新的简历版本。系统只能基于用户已有经历进行表达优化，不允许编造经历。

8.8 简历优化模块

权重：10。

功能包括完整度评分、语句表达优化、项目成果量化建议、技术关键词补强、冗余内容删除、简历风险提示、一键应用部分建议和保存优化报告。

8.9 模拟面试模块

权重：8。

系统根据目标岗位和简历版本生成技术基础题、项目深挖题、系统设计题、行为面试题和反问面试官问题。用户提交文字回答后，系统生成评分、优点、不足、改进建议和参考回答。

8.10 自动投递辅助模块

权重：9。

系统维护用户标准投递信息和常用问答，根据投递站点字段配置生成填表方案。系统提供三种模式：标准信息复制、半自动填表、Playwright 自动化演示。遇到验证码、登录、短信验证时暂停并提示用户手动处理。

8.11 投递进度管理模块

权重：9。

功能包括创建投递记录、自动投递后生成记录、手动添加记录、状态流转、状态历史、笔试/面试时间记录、Offer 信息、拒绝原因、备注、投递看板和提醒。

状态包括：待投递、已投递、简历筛选中、笔试、技术一面、技术二面、HR 面、Offer、已拒绝、已放弃。

8.12 数据统计与求职报告模块

权重：8。

功能包括投递总数、本周投递数量、岗位来源分布、岗位类型分布、城市分布、匹配度区间分布、面试转化率、Offer 转化率、简历版本使用效果、自动投递成功率和 AI 求职周报。

8.13 后台管理模块

权重：6。

功能包括用户管理、简历模板管理、Prompt 模板管理、AI 调用日志、OCR 任务日志、岗位导入任务日志、自动投递站点配置、系统配置、文件管理、运行日志和异常日志查看。

9. 客户端与服务端功能划分

9.1 响应式 Web 前端

只开发一套 Web 前端，适配桌面端、平板和手机浏览器。

桌面端重点支持完整工作流：简历编辑、岗位管理、报告查看、自动投递配置和后台管理。

手机端重点支持轻量场景：查看投递进度、上传截图、快速复制投递信息、查看匹配报告摘要、查看求职周报和通知。

前端页面包括：登录页、注册页、首页仪表盘、文字生成简历页、简历上传解析页、简历列表页、简历编辑器页、模板选择页、用户模板上传页、简历预览导出页、岗位导入页、岗位列表页、岗位详情页、匹配报告页、岗位适配页、简历优化页、模拟面试页、自动投递页、投递看板页、数据统计页、求职报告页、管理员后台页。

9.2 Python 后端服务

后端负责业务逻辑、数据存储、AI 调用、OCR、文件解析、模板渲染、自动投递任务、权限控制和日志监控。

10. 系统架构设计

10.1 技术选型

前端 Web：Vue 3 + TypeScript + Vite + Element Plus + Pinia + Axios + ECharts，使用 CSS 媒体查询和响应式布局适配手机浏览器。

后端：Python 3.11 + FastAPI + SQLAlchemy 2.0 + Pydantic v2 + Alembic。

数据库：MySQL 8。

缓存与任务队列：Redis 7。

异步任务：Celery。用于 AI 长任务、OCR 任务、PDF 导出和自动投递任务。

文件存储：服务器本地文件系统。若时间充足，可扩展 MinIO。

AI API：DeepSeek API，备用 OpenAI API 或通义千问 API。

OCR：PaddleOCR 优先；Tesseract OCR 或云 OCR API 作为备用。

文档解析：python-docx、PyMuPDF、pdfplumber、pypdf。

模板渲染：Jinja2。

PDF 导出：Playwright Python 渲染 HTML 并导出 PDF。

自动投递辅助：Playwright Python。

接口文档：FastAPI 自动生成 OpenAPI / Swagger UI。

部署方式：直接部署在云服务器上，不使用 Docker。Nginx 负责静态文件托管和 API 反向代理，后端由 Gunicorn + Uvicorn Worker 或 Uvicorn + systemd 管理。

10.2 后端模块划分

采用“模块化单体 + 清晰服务边界”方案。代码以 Python package 组织，后续可拆分为独立服务。课程项目阶段在同一 FastAPI 项目内完成开发，并使用 Celery Worker 处理耗时任务。

后端模块包括：

1. core：配置、鉴权、异常、日志、统一响应。
2. db：数据库连接、SQLAlchemy Base、Session 管理、Alembic 迁移。
3. auth：注册、登录、JWT、角色权限。
4. user：用户资料、标准投递信息、常用问答。
5. resume：简历、简历版本、经历、技能、获奖。
6. template：系统模板、用户模板、模板渲染、PDF 导出。
7. file：文件上传、下载、文件元数据。
8. ocr：OCR 任务、图片版 PDF 识别。
9. job：岗位、公司、岗位要求、标签。
10. job_import：文本、URL、文件、截图导入岗位。
11. matching：岗位匹配基础评分。
12. ai：LLM 调用、Prompt 模板、AI 报告、AI 日志。
13. interview：模拟面试、题目、回答、评价。
14. delivery：自动投递档案、站点配置、投递任务、Playwright 执行。
15. application：投递记录、状态流转、状态历史。

- 16. statistics: 统计数据与图表接口。
- 17. report: 求职报告生成与查询。
- 18. notification: 站内通知与提醒。
- 19. admin: 后台管理。

10.3 Python 后端项目结构

```
backend/  
  app/  
    main.py  
  api/  
    router.py  
    deps.py  
  core/  
    config.py  
    security.py  
    response.py  
    exceptions.py  
    logging.py  
  db/  
    base.py  
    session.py  
  modules/  
    auth/  
      router.py  
      service.py  
      repository.py  
      models.py  
      schemas.py  
    user/  
    resume/  
    template/  
    file/  
    ocr/  
    job/  
    job_import/  
    matching/  
    ai/  
    interview/  
    delivery/  
    application/  
    statistics/  
    report/  
    notification/  
    admin/  
  tasks/  
    celery_app.py
```

```
ai_tasks.py
ocr_tasks.py
delivery_tasks.py
export_tasks.py
utils/
  text_extract.py
  pdf_utils.py
  template_utils.py
  validators.py
alembic/
scripts/
  start_backend.sh
  start_worker.sh
  stop_backend.sh
  deploy.sh
logs/
uploads/
requirements.txt
.env.example
gunicorn_conf.py
```

10.4 云服务器部署结构

系统直接部署在云服务器，不使用 Docker。推荐服务器系统为 Ubuntu 22.04 LTS。

服务器组件：

1. Nginx：托管前端 dist，反向代理 /api 到 FastAPI。
2. FastAPI 后端：运行在 127.0.0.1:8000。
3. Celery Worker：运行 OCR、AI、PDF 导出和自动投递任务。
4. MySQL 8：本机安装或云数据库。
5. Redis 7：本机安装，用作缓存和 Celery Broker。
6. PaddleOCR/Tesseract：本机安装 OCR 依赖。
7. Playwright Chromium：本机安装浏览器运行环境。
8. systemd：管理后端服务和 Celery Worker。
9. Git：拉取代码和版本更新。

推荐目录：

```
/opt/jobpilot/  
  backend/  
  frontend/  
  venv/  
  uploads/  
  logs/  
  scripts/  
  .env
```

10.5 认证与鉴权设计

认证方式：JWT。

角色：ROLE_USER、ROLE_ADMIN。

实现方式：

1. 使用 passlib[bcrypt] 加密密码。
2. 使用 python-jose 或 PyJWT 生成和解析 JWT。
3. 使用 FastAPI Depends 实现 get_current_user。
4. 使用 require_roles 和 require_permissions 控制接口访问。
5. 用户只能访问自己的简历、岗位、投递记录和报告。
6. 管理员可以管理模板、Prompt、用户和系统配置。
7. 文件下载必须校验文件归属。
8. AI、OCR 和自动投递接口必须限流并记录日志。

11. 核心数据模型

11.1 核心领域类

User、Role、Permission、UserProfile、DeliveryProfile、DeliveryCommonAnswer、Resume、ResumeVersion、EducationExperience、ProjectExperience、InternshipExperience、ResearchExperience、Skill、Award、ResumeTemplate、ResumeRenderRecord、FileResource、OCRTask、Company、JobPosition、JobRequirement、JobTag、JobImportTask、MatchReport、OptimizeReport、AdaptationRecord、InterviewSession、InterviewQuestion、InterviewAnswer、

InterviewEvaluation、DeliverySite、DeliveryFieldMapping、DeliveryTask、DeliveryTaskLog、ApplicationRecord、ApplicationStatusHistory、StatisticsSnapshot、JobSearchReport、AIAnalysisReport、AICallLog、PromptTemplate、Notification、SystemConfig。

11.2 关键关系

User 与 Resume 一对多。Resume 与 ResumeVersion 一对多。ResumeVersion 组合教育经历、项目经历、实习经历、科研经历、技能和获奖经历。ResumeVersion 使用 ResumeTemplate 渲染。User 与 JobPosition 一对多。JobPosition 与 JobRequirement 一对一。JobPosition 与 JobTag 多对多。ResumeVersion 与 JobPosition 可以生成 MatchReport、AdaptationRecord、InterviewSession。User 与 DeliveryProfile 一对一。DeliveryTask 关联 User、JobPosition、ResumeVersion 和 DeliverySite。ApplicationRecord 关联 User、JobPosition、ResumeVersion 和 DeliveryTask。ApplicationRecord 包含多个 ApplicationStatusHistory。AICallLog 记录 AI 调用过程，OCRTask 记录 OCR 识别过程，JobImportTask 记录岗位导入过程。

12. 数据库表设计

12.1 用户与权限表

sys_user: id、username、password_hash、email、phone、avatar_url、status、created_at、updated_at。

sys_role: id、role_code、role_name、description。

sys_permission: id、permission_code、permission_name、resource_path、method。

sys_user_role: id、user_id、role_id。

sys_role_permission: id、role_id、permission_id。

login_log: id、user_id、ip、user_agent、login_status、created_at。

12.2 用户资料与投递档案表

user_profile: id、user_id、real_name、gender、birthday、school、major、degree、graduation_year、target_position、target_city。

delivery_profile: id、user_id、real_name、phone、email、id_number_masked、school、major、degree、graduation_date、expected_salary、available_date、political_status、english_level、address、self_introduction、created_at、updated_at。

delivery_common_answer: id、user_id、question_key、question_text、answer_text、created_at、updated_at。

12.3 简历表

resume: id、user_id、title、target_position、is_default、created_at、updated_at。

resume_version: id、resume_id、version_name、version_no、source_type、template_id、content_json、completeness_score、created_at。

education_experience: id、resume_version_id、school、major、degree、start_date、end_date、description。

project_experience: id、resume_version_id、project_name、role_name、tech_stack、start_date、end_date、description、achievement。

internship_experience: id、resume_version_id、company_name、position_name、start_date、end_date、description、achievement。

research_experience: id、resume_version_id、title、role_name、description、achievement。

skill: id、resume_version_id、skill_name、skill_level、category。

award: id、resume_version_id、award_name、award_level、award_date、description。

12.4 模板与文件表

resume_template: id、owner_user_id、template_name、template_code、template_type、visibility、template_source、preview_image_url、html_template_path、css_content、placeholder_schema、enabled、created_at、updated_at。

resume_render_record: id、user_id、resume_version_id、template_id、render_file_id、render_status、created_at。

file_resource: id、user_id、file_name、file_path、file_type、file_size、related_type、related_id、created_at。

ocr_task: id、user_id、file_id、task_status、page_count、ocr_text、confidence_avg、error_message、created_at、finished_at。

12.5 岗位表

company: id、company_name、industry、company_size、description。

job_position: id、user_id、company_id、job_title、city、job_type、salary_range、source_type、source_url、source_file_id、deadline、raw_jd、created_at、updated_at。

job_requirement: id、job_id、education_required、experience_required、skill_keywords、responsibility_summary、requirement_summary、parsed_json。

job_tag: id、tag_name。

job_position_tag: id、job_id、tag_id。

job_import_task: id、user_id、source_type、source_url、source_file_id、raw_text、task_status、parsed_count、error_message、created_at、finished_at。

12.6 AI 与匹配表

match_report: id、user_id、resume_version_id、job_id、total_score、skill_score、project_score、education_score、experience_score、preference_score、completeness_score、summary、detail_json、created_at。

optimize_report: id、user_id、resume_version_id、score_before、summary、suggestion_json、created_at。

adaptation_record: id、user_id、source_resume_version_id、target_resume_version_id、job_id、adaptation_summary、created_at。

ai_analysis_report: id、user_id、report_type、related_id、title、content_json、created_at。

ai_call_log: id、user_id、model_name、prompt_type、input_tokens、output_tokens、cost_estimate、status、error_message、duration_ms、created_at。

prompt_template: id、template_code、template_name、template_content、version、enabled、created_at、updated_at。

12.7 模拟面试表

interview_session: id、user_id、job_id、resume_version_id、session_title、difficulty、created_at。

interview_question: id、session_id、question_type、question_text、reference_answer、sort_order。

interview_answer: id、question_id、user_answer、submitted_at。

interview_evaluation: id、answer_id、score、advantages、weaknesses、suggestion、improved_answer、created_at。

12.8 自动投递与投递进度表

delivery_site: id、site_name、site_url、site_type、enabled、description。

delivery_field_mapping: id、site_id、field_key、field_label、selector、value_source、required、sort_order。

delivery_task: id、user_id、job_id、resume_version_id、site_id、target_url、task_status、submit_mode、created_at、finished_at、error_message。

delivery_task_log: id、task_id、step_name、step_status、message、created_at。

application_record: id、user_id、job_id、resume_version_id、delivery_task_id、status、apply_date、next_event_time、remark、created_at、updated_at。

application_status_history: id、application_id、from_status、to_status、changed_at、remark。

12.9 统计与通知表

statistics_snapshot: id、user_id、snapshot_date、total_apply_count、interview_count、offer_count、reject_count、auto_delivery_success_rate、data_json。

job_search_report: id、user_id、report_type、start_date、end_date、title、content_json、created_at。

notification: id、user_id、title、content、type、read_status、created_at。

system_config: id、config_key、config_value、description。

13. AI、OCR 与文档解析设计

13.1 AI 服务总原则

1. AI 只做辅助，不替用户做最终决策。
2. AI 不允许编造学历、项目、实习、获奖或证书。
3. AI 生成内容必须可追溯到用户已有输入。
4. AI API Key 只能保存在服务器 .env 中，前端不得保存。
5. 所有 AI 调用必须记录日志。
6. 所有 AI 接口必须进行用户限流。
7. 所有 AI 输出必须由用户确认后才能应用到简历或投递内容。

13.2 Python AI 服务封装

核心类：AIClient、PromptService、AILogService。

AIClient 提供 `async chat`、`async chat_json`、`async chat_with_usage` 方法。

PromptService 负责从 `prompt_template` 表读取模板并使用 Jinja2 渲染变量。

AILogService 负责记录模型名称、Prompt 类型、输入输出 token、耗时、调用状态和错误信息。

13.3 AI 能力清单

1. 文字生成简历：POST `/api/ai/resumes/generate-from-text`。
2. 简历文件结构化解析：POST `/api/ai/resumes/parse-file`。
3. OCR 文本结构化解析：POST `/api/ai/resumes/parse-ocr-text`。
4. 岗位 JD 解析：POST `/api/ai/jobs/parse`。
5. 岗位匹配报告：POST `/api/ai/matching/analyze`。
6. 简历岗位适配：POST `/api/ai/resumes/adapt-to-job`。
7. 简历优化：POST `/api/ai/resumes/optimize`。
8. 模拟面试题生成：POST `/api/ai/interviews/questions`。
9. 面试回答评价：POST `/api/ai/interviews/evaluate-answer`。
10. 求职周报生成：POST `/api/ai/reports/weekly`。

13.4 OCR 解析流程

1. 用户上传 PDF、PNG、JPG 或 JPEG。
2. file 模块保存文件并生成 file_resource。
3. 如果是 DOCX，使用 python-docx 提取文本。
4. 如果是 PDF，先用 PyMuPDF 或 pdfplumber 提取文本。
5. 如果 PDF 文本为空或过短，判定为图片版 PDF。
6. 图片版 PDF 使用 PyMuPDF 将每页渲染为图片。
7. OCR 模块调用 PaddleOCR 或 Tesseract 识别图片文本。
8. OCR 文本保存到 ocr_task。
9. AI 模块将 OCR 文本解析为结构化简历或岗位信息。
10. 用户确认后保存结果。

13.5 Prompt 基本规则

所有 Prompt 必须包含：角色设定、输入数据说明、输出 JSON Schema、禁止编造规则、输出语言要求和异常处理规则。若无法确定字段，输出空字符串或空数组，不得编造。

14. 岗位信息获取与自动投递设计

14.1 岗位导入接口

POST /api/jobs/import/text：通过用户粘贴文本导入岗位。

POST /api/jobs/import/url：通过公开 URL 尝试解析岗位。

POST /api/jobs/import/file：通过文件导入岗位。

POST /api/jobs/import/image：通过招聘截图 OCR 导入岗位。

POST /api/jobs/import/batch-text：批量粘贴多个岗位文本并解析。

GET /api/jobs/import/tasks/{id}：查询导入任务状态。

14.2 自动投递模式

模式一：标准信息复制。系统生成填表清单，用户复制粘贴。

模式二：半自动填表。系统根据字段映射填充可识别字段，用户检查后手动提交。

模式三：Playwright 自动化演示。对课程项目自建模拟招聘页面或配置好的站点，系统自动打开页面、填充字段、上传简历并截图。遇到验证码、登录或短信验证时暂停。

14.3 自动投递接口

GET /api/delivery/profiles/me。

PUT /api/delivery/profiles/me。

GET /api/delivery/common-answers。

POST /api/delivery/common-answers。

POST /api/delivery/tasks。

POST /api/delivery/tasks/{id}/preview。

POST /api/delivery/tasks/{id}/execute。

GET /api/delivery/tasks/{id}。

GET /api/delivery/tasks/{id}/logs。

POST /api/admin/delivery-sites。

POST /api/admin/delivery-sites/{id}/field-mappings。

14.4 Playwright Python 执行流程

1. Celery Worker 接收 delivery_task_id。
2. 查询目标 URL、字段映射和用户投递档案。
3. 使用 Playwright Python 启动 Chromium。
4. page.goto(target_url)。
5. 遍历字段映射，执行 page.fill(selector, value)。
6. 若存在文件上传字段，上传导出的 PDF 简历。

7. 截图并保存。
8. 写入 delivery_task_log。
9. 若遇到验证码或登录页面，任务状态变为 WAITING_USER。
10. 用户确认后创建或更新 application_record。

15. 投递状态机

状态流转：待投递 -> 已投递 -> 简历筛选中 -> 笔试 -> 技术一面 -> 技术二面 -> HR 面 -> Offer。

任意非终止状态可以转为已放弃。筛选、笔试或面试失败可以转为已拒绝。终止状态为 Offer、已拒绝、已放弃。

每次状态变更必须写入 application_status_history。

16. 接口总览

16.1 认证接口

POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me, PUT /api/auth/password。

16.2 用户资料接口

GET /api/users/me, PUT /api/users/me, GET /api/users/profile, PUT /api/users/profile。

16.3 文件与 OCR 接口

POST /api/files/upload, GET /api/files/{id}, GET /api/files/{id}/download, POST /api/ocr/extract-text, GET /api/ocr/tasks/{id}。

16.4 简历接口

POST /api/resumes, GET /api/resumes, GET /api/resumes/{id}, PUT /api/resumes/{id}, DELETE /api/resumes/{id}, POST /api/resumes/{id}/versions, GET /api/resumes/{id}/versions, GET /api/resume-versions/{version_id}, PUT /api/resume-versions/{version_id}, DELETE /api/resume-versions/{version_id}, POST /api/resume-versions/{version_id}/copy。

16.5 简历模板接口

GET /api/resume-templates, GET /api/resume-templates/my, POST /api/resume-templates/upload, POST /api/resume-templates/{id}/copy, PUT /api/resume-templates/{id}, DELETE /api/resume-templates/{id}, POST /api/resume-versions/{version_id}/render, POST /api/resume-versions/{version_id}/export-pdf, POST /api/admin/resume-templates, PUT /api/admin/resume-templates/{id}。

16.6 岗位接口

POST /api/jobs, GET /api/jobs, GET /api/jobs/{id}, PUT /api/jobs/{id}, DELETE /api/jobs/{id}, POST /api/jobs/{id}/parse, GET /api/jobs/{id}/requirements, POST /api/jobs/{id}/tags。

16.7 岗位导入接口

POST /api/jobs/import/text, POST /api/jobs/import/url, POST /api/jobs/import/file, POST /api/jobs/import/image, POST /api/jobs/import/batch-text, GET /api/jobs/import/tasks/{id}。

16.8 AI 接口

POST /api/ai/resumes/generate-from-text, POST /api/ai/resumes/parse-file, POST /api/ai/resumes/parse-ocr-text, POST /api/ai/resumes/optimize, POST /api/ai/resumes/adapt-to-job, POST /api/ai/jobs/parse, POST /api/ai/matching/analyze, POST /api/ai/interviews/questions, POST /api/ai/interviews/evaluate-answer, POST /api/ai/reports/weekly。

16.9 模拟面试接口

POST /api/interviews/sessions, GET /api/interviews/sessions, GET /api/interviews/sessions/{id}, POST /api/interviews/sessions/{id}/questions, POST /api/interviews/questions/{id}/answers, GET

/api/interviews/answers/{id}/evaluation。

16.10 自动投递接口

GET /api/delivery/profiles/me, PUT /api/delivery/profiles/me, GET /api/delivery/common-answers, POST /api/delivery/common-answers, PUT /api/delivery/common-answers/{id}, DELETE /api/delivery/common-answers/{id}, POST /api/delivery/tasks, POST /api/delivery/tasks/{id}/preview, POST /api/delivery/tasks/{id}/execute, GET /api/delivery/tasks, GET /api/delivery/tasks/{id}, GET /api/delivery/tasks/{id}/logs。

16.11 投递进度接口

POST /api/applications, GET /api/applications, GET /api/applications/{id}, PUT /api/applications/{id}, DELETE /api/applications/{id}, POST /api/applications/{id}/status, GET /api/applications/{id}/status-history, GET /api/applications/kanban。

16.12 统计报告接口

GET /api/statistics/overview, GET /api/statistics/applications, GET /api/statistics/jobs, GET /api/statistics/matches, GET /api/reports, POST /api/reports/weekly, GET /api/reports/{id}。

16.13 管理员接口

GET /api/admin/users, PUT /api/admin/users/{id}/status, GET /api/admin/ai-logs, GET /api/admin/ocr-logs, GET /api/admin/system-logs, GET /api/admin/prompt-templates, POST /api/admin/prompt-templates, PUT /api/admin/prompt-templates/{id}, GET /api/admin/delivery-sites, POST /api/admin/delivery-sites, PUT /api/admin/delivery-sites/{id}。

17. 云服务器部署方案

17.1 服务器要求

推荐配置：2 核 4GB 起步，推荐 4 核 8GB。操作系统 Ubuntu 22.04 LTS。若部署 PaddleOCR，建议 4GB 内存以上。

开放端口：22、80、443。后端 8000、MySQL 3306、Redis 6379 默认仅本机访问，不对公网开放。

17.2 服务器软件安装

需要安装：

1. Python 3.11。
2. python3.11-venv。
3. MySQL 8。
4. Redis 7。
5. Nginx。
6. Git。
7. Node.js 20。
8. Playwright Chromium 运行依赖。
9. PaddleOCR 或 Tesseract OCR。
10. systemd 服务配置。

17.3 部署目录

```
/opt/jobpilot/  
  backend/  
  frontend/  
  venv/  
  uploads/  
  logs/  
  scripts/  
  .env
```


17.4 后端部署步骤

1. SSH 登录服务器。
2. 创建目录 /opt/jobpilot。
3. git clone 后端代码到 /opt/jobpilot/backend。
4. 创建 Python 虚拟环境：python3.11 -m venv /opt/jobpilot/venv。
5. 激活虚拟环境并安装依赖：pip install -r requirements.txt。
6. 创建 MySQL 数据库 jobpilot。
7. 配置 /opt/jobpilot/.env。
8. 执行 Alembic 迁移：alembic upgrade head。
9. 初始化管理员账号和基础模板数据。
10. 使用 systemd 启动 FastAPI 服务。
11. 使用 systemd 启动 Celery Worker。

17.5 systemd 服务示例

后端服务 jobpilot-backend.service：

```
[Unit]
Description=JobPilot FastAPI Backend
After=network.target mysql.service redis-server.service

[Service]
User=www-data
WorkingDirectory=/opt/jobpilot/backend
EnvironmentFile=/opt/jobpilot/.env
ExecStart=/opt/jobpilot/venv/bin/gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 2 -b
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Celery Worker 服务 jobpilot-worker.service：

```
[Unit]
Description=JobPilot Celery Worker
After=network.target redis-server.service

[Service]
User=www-data
WorkingDirectory=/opt/jobpilot/backend
EnvironmentFile=/opt/jobpilot/.env
ExecStart=/opt/jobpilot/venv/bin/celery -A app.tasks.celery_app worker -l info
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

17.6 前端部署步骤

1. 在本地或服务器安装 Node.js 20。
2. 进入 frontend 目录。
3. 执行 npm install。
4. 执行 npm run build。
5. 将 dist 目录复制到 /opt/jobpilot/frontend/dist。
6. 配置 Nginx 静态文件路径。

17.7 Nginx 配置示例

```
server {  
    listen 80;  
    server_name _;  
  
    root /opt/jobpilot/frontend/dist;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://127.0.0.1:8000/api/;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
  
    location /uploads/ {  
        alias /opt/jobpilot/uploads/;  
        autoindex off;  
    }  
}
```

17.8 日志与故障定位

日志位置：

1. Nginx 日志：/var/log/nginx/access.log、/var/log/nginx/error.log。
2. 后端日志：/opt/jobpilot/logs/backend.log。
3. Worker 日志：/opt/jobpilot/logs/worker.log。
4. systemd 日志：journalctl -u jobpilot-backend -f, journalctl -u jobpilot-worker -f。
5. MySQL 日志和 Redis 日志按服务器默认路径。

故障定位流程：先看浏览器 Network，再看 Nginx error.log，再看后端日志，再查数据库和 Redis 状态。AI、OCR、自动投递相关问题查 ai_call_log、ocr_task、delivery_task_log 和 Celery Worker 日志。

18. 项目开发计划

第 1 周：需求、原型、架构、基础工程

完成项目说明、功能分解树、用例图、领域类图、数据库 ER 图、前端原型、FastAPI 基础工程、Git 仓库和分支规范。

第 2 周：用户、简历、文件、岗位核心功能

完成认证权限、用户资料、简历 CRUD、简历版本管理、文件上传、DOCX/PDF 文本解析、岗位管理、岗位文本导入。

第 3 周：OCR、AI、模板、匹配

完成 OCR 简历解析、图片版 PDF 解析、文字生成简历、岗位 JD 解析、简历优化、岗位适配、岗位匹配报告、模板切换、用户模板上传和 PDF 导出。

第 4 周：模拟面试、自动投递、投递管理、统计

完成模拟面试、Playwright 自动填表演示、投递状态机、投递看板、数据统计、求职周报、响应式 Web 适配。

第 5 周：云服务器部署、测试、文档完善

完成云服务器直接部署、Nginx 反向代理、systemd 服务、运行截图、接口文档、三份课程文档、代码量统计、开发记录、Git Commit 记录和最终压缩包。

第二部分：四人小组工作文档

成员 A：组长、Python 后端架构、认证权限、云服务器部署

A.1 角色定位

成员 A 是项目组长和后端基础架构负责人，负责项目进度、Git 管理、FastAPI 工程框架、认证鉴权、管理员模块、云服务器部署和系统设计文档中的架构部分。

建议贡献占比：30%。

A.2 技术栈

Python 3.11、FastAPI、SQLAlchemy 2.0、Pydantic v2、Alembic、JWT、passlib[bcrypt]、MySQL、Redis、Gunicorn、Uvicorn、Nginx、systemd、Git。

A.3 具体开发任务

1. 初始化 backend 工程结构。
2. 编写 app/main.py，注册全局路由、中间件和异常处理。
3. 编写 app/core/config.py，读取 .env 配置。
4. 编写 app/db/session.py，配置 SQLAlchemy Engine 和 Session。
5. 配置 Alembic 迁移。
6. 实现统一响应 ApiResponse。
7. 实现 BusinessException 和全局异常处理。
8. 实现请求日志中间件。
9. 实现 sys_user、sys_role、sys_permission 等 ORM 模型。
10. 实现注册、登录、获取当前用户、修改密码接口。
11. 实现 JWT 生成、解析、刷新和鉴权依赖。
12. 实现 require_roles、require_permissions。
13. 实现管理员用户管理接口。
14. 实现系统配置接口。
15. 实现 AI、OCR、系统日志后台查询接口。

16. 编写服务器部署脚本。
17. 配置 Nginx。
18. 配置 systemd 后端服务和 Celery Worker 服务。
19. 完成云服务器上线和测试账号初始化。

A.4 负责接口

POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me, PUT /api/auth/password, GET /api/users/me, PUT /api/users/me, GET /api/admin/users, PUT /api/admin/users/{id}/status, GET /api/admin/ai-logs, GET /api/admin/ocr-logs, GET /api/admin/system-logs, GET /api/admin/system-configs, PUT /api/admin/system-configs/{key}。

A.5 文档任务

负责《系统设计文档》中的技术选型、总体架构、后端模块划分、认证鉴权、云服务器部署、日志监控、安装与卸载方法。负责《项目管理文档》中的成员分工、Git 规范、开发计划、部署记录和代码量统计。

成员 B：响应式 Web 前端、简历编辑器、模板预览、统计可视化

B.1 角色定位

成员 B 负责 Web 前端主应用，包括求职者端、管理员端、响应式适配、简历编辑器、模板预览、岗位导入页面、匹配报告展示、投递看板和统计图表。

建议贡献占比：25%。

B.2 技术栈

Vue 3、TypeScript、Vite、Element Plus、Pinia、Vue Router、Axios、ECharts、SCSS、CSS Media Query、html2canvas/html2pdf.js 可选、Monaco Editor 可选。

B.3 具体开发任务

1. 初始化 frontend 工程。
2. 配置 Axios 请求封装和 Token 拦截器。
3. 配置路由和权限守卫。
4. 配置 Pinia userStore。
5. 实现响应式布局，桌面端侧边栏，手机端顶部导航或底部操作栏。
6. 登录注册页面。
7. 首页仪表盘。
8. 文字生成简历页面。
9. 简历上传解析页面，支持 PDF、DOCX、图片上传。
10. 简历列表和版本管理页面。
11. 简历编辑器页面。
12. 模板选择与用户模板上传页面。
13. 简历预览和导出页面。
14. 岗位导入页面，支持文本、URL、文件、截图。
15. 岗位列表和详情页面。
16. 匹配报告页面。
17. 简历优化和岗位适配页面。
18. 模拟面试页面。
19. 自动投递页面。
20. 投递看板页面。
21. 数据统计页面。
22. 求职报告页面。
23. 管理员用户管理、模板管理、Prompt 管理、AI 日志、OCR 日志、投递站点配置页面。
24. 运行截图整理。

B.4 负责接口对接

负责对接所有前端页面接口，重点包括 `/api/auth`、`/api/resumes`、`/api/resume-templates`、`/api/jobs`、`/api/jobs/import`、`/api/ai`、`/api/interviews`、`/api/delivery`、`/api/applications`、`/api/statistics`、`/api/reports`、`/api/admin`。

B.5 响应式要求

1. 桌面端宽度 1024px 以上使用完整布局。
2. 平板端 768px 到 1023px 使用收缩菜单。

3. 手机端 767px 以下使用单列卡片流。
4. 简历编辑器手机端改为分步骤编辑。
5. 投递看板手机端改为状态分组列表。
6. 图表手机端纵向排列。

B.6 文档任务

负责《系统需求文档》中的原型设计、客户端功能说明、运行截图说明和功能分解树配图。负责《系统设计文档》中的前端架构、响应式设计和页面截图。

成员 C：简历、模板、岗位、匹配、投递进度业务后端

C.1 角色定位

成员 C 负责核心业务后端，包括简历数据、用户模板、岗位数据、岗位导入、岗位匹配基础算法、投递进度、状态机和统计基础接口。

建议贡献占比：25%。

C.2 技术栈

Python 3.11、FastAPI、SQLAlchemy 2.0、Pydantic v2、Alembic、MySQL、Redis、Jinja2、PyMuPDF、python-docx。

C.3 具体开发任务

1. 实现 Resume、ResumeVersion、EducationExperience、ProjectExperience、InternshipExperience、ResearchExperience、Skill、Award ORM 模型和 Schema。
2. 实现简历 CRUD、版本管理、复制版本、删除版本。
3. 实现简历完整度评分。
4. 实现 ResumeTemplate、ResumeRenderRecord 模型。
5. 实现系统模板查询、用户模板上传、模板复制、模板删除、模板渲染、PDF 导出任务创建。

6. 实现模板安全校验，禁止 script、iframe 和外链 JS。
7. 实现 Company、JobPosition、JobRequirement、JobTag、JobImportTask 模型。
8. 实现岗位 CRUD、岗位标签、岗位搜索筛选。
9. 实现岗位导入任务：文本、URL、文件、截图、批量文本。
10. 实现岗位来源记录和岗位去重。
11. 实现基础匹配算法。
12. 实现 MatchReport 存储与查询。
13. 实现 ApplicationRecord、ApplicationStatusHistory 模型。
14. 实现投递记录 CRUD。
15. 实现投递状态机和状态历史。
16. 实现统计 overview、applications、jobs、matches 接口。

C.4 负责接口

POST /api/resumes, GET /api/resumes, GET /api/resumes/{id}, PUT /api/resumes/{id}, DELETE /api/resumes/{id}, POST /api/resumes/{id}/versions, GET /api/resumes/{id}/versions, GET /api/resume-versions/{version_id}, PUT /api/resume-versions/{version_id}, POST /api/resume-versions/{version_id}/copy, GET /api/resume-templates, GET /api/resume-templates/my, POST /api/resume-templates/upload, POST /api/resume-versions/{version_id}/render, POST /api/resume-versions/{version_id}/export-pdf, POST /api/jobs, GET /api/jobs, GET /api/jobs/{id}, PUT /api/jobs/{id}, DELETE /api/jobs/{id}, POST /api/jobs/import/text, POST /api/jobs/import/url, POST /api/jobs/import/file, POST /api/jobs/import/image, POST /api/matching/calculate, GET /api/matching/reports/{id}, POST /api/applications, GET /api/applications, POST /api/applications/{id}/status, GET /api/applications/kanban, GET /api/statistics/overview。

C.5 文档任务

负责《系统需求文档》中的领域模型、领域类图、包图、投递状态机图、功能依赖关系。负责《系统设计文档》中的数据库设计、核心业务流程、岗位导入设计、匹配算法设计和投递状态机设计。

成员 D：AI 服务、OCR、自动投递 Worker、求职报告、文档统筹

D.1 角色定位

成员 D 负责系统智能化能力、OCR 能力、自动投递执行能力、AI 求职报告和文档整合。

建议贡献占比：20%。

D.2 技术栈

Python 3.11、FastAPI、Celery、Redis、httpx、Pydantic、DeepSeek API/OpenAI API、PaddleOCR、Tesseract OCR、PyMuPDF、python-docx、Playwright Python、Jinja2、Markdown/Word 文档排版。

D.3 具体开发任务

1. 实现 AIClient，封装 LLM API 调用。
2. 实现 PromptService，读取和渲染 Prompt 模板。
3. 实现 AILogService，记录 AI 调用日志。
4. 编写文字生成简历 Prompt。
5. 编写简历解析 Prompt。
6. 编写岗位 JD 解析 Prompt。
7. 编写简历优化 Prompt。
8. 编写岗位适配 Prompt。
9. 编写岗位匹配解释 Prompt。
10. 编写模拟面试题生成 Prompt。
11. 编写面试回答评价 Prompt。
12. 编写求职周报 Prompt。
13. 实现 OCRTask 和 OCR 服务。
14. 实现图片版 PDF 转图片和 OCR 识别。
15. 实现 OCR 文本转结构化简历。
16. 实现招聘截图 OCR 转岗位。
17. 实现 Celery AI 任务、OCR 任务、PDF 导出任务和自动投递任务。
18. 实现 Playwright Python 自动填表 Worker。

19. 实现 DeliveryTask、DeliveryTaskLog 的执行逻辑。
20. 实现求职周报生成与保存。
21. 整合三份课程文档。

D.4 负责接口

POST /api/ai/resumes/generate-from-text, POST /api/ai/resumes/parse-file, POST /api/ai/resumes/parse-ocr-text, POST /api/ai/jobs/parse, POST /api/ai/matching/analyze, POST /api/ai/resumes/optimize, POST /api/ai/resumes/adapt-to-job, POST /api/ai/interviews/questions, POST /api/ai/interviews/evaluate-answer, POST /api/ai/reports/weekly, POST /api/ocr/extract-text, GET /api/ocr/tasks/{id}, GET /api/delivery/profiles/me, PUT /api/delivery/profiles/me, POST /api/delivery/tasks, POST /api/delivery/tasks/{id}/preview, POST /api/delivery/tasks/{id}/execute, GET /api/delivery/tasks/{id}/logs。

D.5 文档任务

负责《系统设计文档》中的 AI 服务设计、OCR 设计、自动投递设计、Prompt 设计和技术关键点。负责《项目管理文档》中的 AI 工具使用情况报告、API 调用配置记录、服务器部署记录汇总、开发记录汇总和最终文档排版检查。

第三部分：课程文档拆分建议

1. 《系统需求文档》目录

项目背景、项目目标、领域边界、目标用户、竞品分析、岗位来源说明、功能需求总览、功能分解树、功能优先级与依赖关系、用例图、权限设计、客户端与服务端功能划分、原型设计、运行截图说明、领域类图、包图、活动图：文字生成简历、活动图：OCR 简历解析、顺序图：岗位匹配分析、状态机图：投递状态流转、开发计划。

2. 《系统设计文档》目录

技术选型、系统总体架构、Python 后端模块划分、前端响应式架构设计、数据库设计、认证与鉴权设计、文件解析与 OCR 设计、AI 服务设计、简历模板渲染设计、岗位导入设计、岗位匹配算法设计、自

动投递模块设计、投递状态机设计、高并发与性能设计、日志监控与故障定位、云服务器部署方案、运行与开发环境、安装与卸载方法、系统规模与代码量统计。

3. 《项目管理文档》 目录

小组成员信息、成员分工、贡献占比、Git 仓库说明、分支管理规范、Commit 规范、迭代计划、每周开发记录、Issue 记录、分支合并历史、AI 工具使用情况报告、服务器租用与部署记录、API 调用配置记录、测试记录、代码量统计、项目总结。

第四部分： 最终提交清单

压缩包结构建议：

JobPilot_智能简历求职系统_第X组.zip

```
source_code/  
  backend/  
  frontend/  
  deploy_scripts/  
  sql/  
executable_or_deploy/  
  requirements.txt  
  gunicorn_conf.py  
  nginx_jobpilot.conf  
  jobpilot-backend.service  
  jobpilot-worker.service  
  start_backend.sh  
  start_worker.sh  
documents/  
  系统需求文档.docx  
  系统设计文档.docx  
  项目管理文档.docx  
  UML图汇总.pdf  
  接口文档.pdf  
  运行截图.pdf  
sql/  
  init.sql  
  demo_data.sql  
README.md  
项目说明.txt
```

测试账号：普通用户 student / 123456，管理员 admin / 123456。

第五部分：风险与应对

风险 1：AI 输出 JSON 不稳定。应对：Prompt 中强制 JSON Schema，后端使用 Pydantic 校验，失败时重试或返回可编辑文本。

风险 2：OCR 识别质量不稳定。应对：支持用户预览 OCR 文本并手动修正；优先使用清晰图片；保留原文件和识别文本。

风险 3：URL 岗位解析失败。应对：不承诺所有网站可解析，提供文本粘贴、截图导入和文件导入作为兜底。

风险 4：自动投递真实网站兼容性差。应对：以半自动填表和模拟站点演示为主，不绕过验证码和登录限制。

风险 5：服务器环境复杂。应对：第 4 周前完成服务器部署；使用 systemd 固化启动方式；保留本地运行方案。

风险 6：代码量不足。应对：完整实现 Python 后端、响应式 Web、AI、OCR、Playwright、后台管理、日志和统计，不靠无意义代码堆砌。

第六部分：代码量目标

团队项目目标总代码量：55000 到 65000 行。

建议分布：

1. 后端 Python：25000 到 30000 行。
2. 前端 Web：22000 到 26000 行。
3. AI、OCR、Playwright 任务代码：6000 到 9000 行。
4. SQL、部署脚本、配置：2000 到 3000 行。

统计命令：

```
cloc backend frontend deploy_scripts sql \
  --exclude-dir=node_modules,.venv,__pycache__,dist,.git,alembic/versions
```

第七部分：建议演示流程

1. 登录普通用户账号。
2. 使用文字描述生成简历。
3. 上传文本型 PDF 或 DOCX 简历并解析为结构化简历。
4. 上传图片版 PDF 或截图简历，展示 OCR 识别和结构化解析。
5. 打开简历编辑器，修改项目经历。
6. 上传或选择简历模板，渲染并导出 PDF。
7. 通过文本或截图导入岗位。

8. 系统解析岗位要求。
9. 选择简历进行岗位匹配。
10. 查看匹配报告。
11. 生成岗位适配版简历。
12. 对简历进行 AI 优化。
13. 发起模拟面试并回答一道题。
14. 查看 AI 评价。
15. 创建自动投递任务。
16. 预览自动填写字段。
17. 执行半自动投递或模拟站点自动填表。
18. 查看投递记录和投递状态历史。
19. 在投递看板更新状态。
20. 查看数据统计。
21. 生成求职周报。
22. 切换管理员账号查看 AI 日志、OCR 日志、模板配置和系统配置。

第八部分：结论

本项目以“智能简历生成与求职投递管理”为核心，围绕求职者真实工作流构建完整业务闭环。系统既包含传统软件工程中的用户管理、权限控制、数据库建模、文件管理、模板渲染、状态机、统计报表、日志监控和云服务器部署，也包含大语言模型驱动的文字生成简历、简历解析、岗位适配、简历优化、岗位匹配、模拟面试和求职报告生成，并通过 OCR 支持图片版 PDF 与截图解析，通过 Playwright Python 提供自动填表辅助。

项目采用 Python + FastAPI 后端，直接部署在云服务器上，减少 Docker 学习和运维成本，便于小组快速实现 OCR、AI、PDF 解析和浏览器自动化等功能。四名成员的分工覆盖后端架构、前端交互、核心业务、AI/OCR/自动投递和文档工作，能够支撑团队项目规模和课程文档要求。